MMC5

From NESdev Wiki

The **Nintendo MMC5** is a mapper ASIC used in Nintendo's ExROM Game Pak boards. All MMC5 boards are assigned to **mapper 5**.

MMC5 ExROM	
Company	Nintendo, Koei, others
Games	15 in NesCartDB (https://nescartdb.com/search/advanced?ines=5)
Complexity	ASIC
Boards	EKROM, ELROM, ETROM, EWROM
Pinout	MMC5 pinout
PRG ROM capacity	1024K
PRG ROM window	8K, 16K, or 32K
PRG RAM capacity	128K
PRG RAM window	8K ($6000-$DFFF), 16K (only $8000-$BFFF at PRG mode 1/2)
CHR capacity	1024K
CHR window	1K, 2K, 4K, or 8K
Nametable arrangement	arbitrary, up to 3 source nametables (plus fill mode)
Bus conflicts	No
IRQ	Yes
Audio	Yes
iNES mappers	005

NESCartDB
iNES 005 (https://nescartdb.com/search/advanced?ines=5)
ExROM (https://nescartdb.com/search/advanced?unif_op=LIKE+`%25%40%25`&unif=-E%25ROM)
EKROM (https://nescartdb.com/search/advanced?unif_op=LIKE+`%25%40%25`&unif=EKROM)
ELROM (https://nescartdb.com/search/advanced?unif_op=LIKE+`%25%40%25`&unif=ELROM)
ETROM (https://nescartdb.com/search/advanced?unif_op=LIKE+`%25%40%25`&unif=ETROM)
EWROM (https://nescartdb.com/search/advanced?unif_op=LIKE+`%25%40%25`&unif=EWROM)

Example games:

- *Castlevania 3*
- *Just Breed*
- *Uncharted Waters*
- *Romance of the Three Kingdoms II*
- *Laser Invasion*
- *Metal Slader Glory*
- *Uchuu Keibitai SDF*
- *Shin 4 Nin Uchi Mahjong - Yakuman Tengoku*
- *Bandit Kings of Ancient China*

The first game to use this chip (*Nobunaga's Ambition II*) was released in February 1990. The date codes on components on early released cartridges (http://bootgod.dyndns.org:7777/profile.php?id=3169) show that manufacturing had started at the end of 1989.

A later **MMC5A** revision was introduced with a few extra features, but all released games do not rely on these features and are compatible with the original MMC5.

Contents

- 1 Overview
- 2 Banks
  - 2.1 PRG mode 0
  - 2.2 PRG mode 1
  - 2.3 PRG mode 2
  - 2.4 PRG mode 3
  - 2.5 CHR mode 0
  - 2.6 CHR mode 1
  - 2.7 CHR mode 2
  - 2.8 CHR mode 3
- 3 Registers
  - 3.1 Sound ($5000-$5015)
  - 3.2 NES internal state monitoring
    - 3.2.1 8x16 mode enable ($2000 = PPUCTRL)
    - 3.2.2 PPU Data Substitution Enable ($2001 = PPUMASK)
    - 3.2.3 Scanline Counter Reset ($2002 = PPUSTATUS, read only)
    - 3.2.4 Unknown ($2005 = PPUSCROLL)
    - 3.2.5 Unknown ($2006 = PPUADDR, MMC5A only)
    - 3.2.6 Scanline Counter Reset ($4014 = OAMDMA)
  - 3.3 Configuration
    - 3.3.1 PRG mode ($5100)
    - 3.3.2 CHR mode ($5101)
    - 3.3.3 PRG RAM Protect 1 ($5102)
    - 3.3.4 PRG RAM Protect 2 ($5103)
    - 3.3.5 Internal extended RAM mode ($5104)
      - 3.3.5.1 Extended attributes
    - 3.3.6 Nametable mapping ($5105)
    - 3.3.7 Fill-mode tile ($5106)
    - 3.3.8 Fill-mode color ($5107)
  - 3.4 PRG Bankswitching ($5113-$5117)
    - 3.4.1 Separate PRG-ROM and PRG-RAM Address Buses
    - 3.4.2 PRG-RAM configurations
    - 3.4.3 Other PRG-RAM notes
  - 3.5 CHR Bankswitching ($5120-$5130)
    - 3.5.1 CHR select $5120-$512B
    - 3.5.2 Upper CHR Bank bits ($5130)
  - 3.6 Other Registers
    - 3.6.1 Vertical Split Mode ($5200)
      - 3.6.1.1 Method of Operation
      - 3.6.1.2 Horizontal Behavior
      - 3.6.1.3 Vertical Behavior
    - 3.6.2 Vertical Split Scroll ($5201)
    - 3.6.3 Vertical Split Bank ($5202)
    - 3.6.4 IRQ Scanline Compare Value ($5203)
    - 3.6.5 Scanline IRQ Status ($5204, read/write)
      - 3.6.5.1 Write
      - 3.6.5.2 Read
    - 3.6.6 Unsigned 8x8 to 16 Multiplier ($5205, $5206 read/write)
      - 3.6.6.1 Write
      - 3.6.6.2 Read
  - 3.7 Internal extended RAM ($5C00-$5FFF, read/write)

- 4 MMC5A
  - 4.1 MMC5A Registers
    - 4.1.1 CL3 / SL3 Data Direction and Output Data Source (MMC5A: $5207 write only)
    - 4.1.2 CL3 / SL3 Status (MMC5A: $5208 read/write)
      - 4.1.2.1 Write
      - 4.1.2.2 Read
    - 4.1.3 16-bit Hardware Timer with IRQ (MMC5A: $5209 read/write, $520A write)
      - 4.1.3.1 Read
      - 4.1.3.2 Write
    - 4.1.4 Unknown Address Range (MMC5A: $5800-$5BFF, write only)
- 5 Scanline Detection and Scanline IRQ
- 6 Hardware
- 7 References
- 8 External links

Overview

The MMC5 is the most elaborate mapper ASIC Nintendo made for the NES and Famicom.

It supports a kitchen sink of features, including:

- 4 PRG ROM switching modes
- 4 CHR ROM switching modes
- Up to 128KB of WRAM, mappable not only at $6000-$7FFF but also within $8000-$DFFF
  - Supports either one chip (up to 128KB) or two chips (up to 32KB each)
- An 8 bit by 8 bit multiplier with a 16 bit result for performing quick calculations
- Scanline detection with counter and configurable IRQ
- Frame detection with status bit
- The ability to use different CHR banks for background and 8x16 sprites (allowing 256 unique 8x16 sprite tiles, independent of the background).
- 1024 bytes of on-chip memory, which can be used for 4 different purposes:
  - An extra general-use nametable
  - Attribute and tile index expansion - address 16384 background tiles at once, and allow each individual 8x8 tile to have its own palette setting
  - Vertical split-screen
  - Extra RAM for storing program variables
- Three extra sound channels
  - Two pulse channels, identical to those in the NES APU (except lacking pitch sweeps).
  - An 8-bit RAW PCM channel
- A 'fill mode' nametable, which can be instantly set to contain a specific tile in a specific color (useful for screen transitions)
- System reset detection
  - Triggered by a positive or negative gap in M2 of at least 11.2 usec.
  - Also triggered and latched by absence of AVcc.
  - After reapplying AVcc, another gap in M2 is sometimes necessary to clear the latch.
  - This feature resets some, but not all, states of the MMC5.
  - The PRG RAM +CE pin is a direct reflection of system reset detection state.
  - Reduces the need to hold the Reset Button while turning off the power in games with battery save.[1]

Banks

The MMC5 provides 4 distinct banking modes for both PRG ROM and CHR ROM.

PRG mode 0

- CPU $6000-$7FFF: 8 KB switchable PRG RAM bank
- CPU $8000-$FFFF: 32 KB switchable PRG ROM bank

PRG mode 1

- CPU $6000-$7FFF: 8 KB switchable PRG RAM bank
- CPU $8000-$BFFF: 16 KB switchable PRG ROM/RAM bank
- CPU $C000-$FFFF: 16 KB switchable PRG ROM bank

PRG mode 2

- CPU $6000-$7FFF: 8 KB switchable PRG RAM bank
- CPU $8000-$BFFF: 16 KB switchable PRG ROM/RAM bank
- CPU $C000-$DFFF: 8 KB switchable PRG ROM/RAM bank
- CPU $E000-$FFFF: 8 KB switchable PRG ROM bank

PRG mode 3

- CPU $6000-$7FFF: 8 KB switchable PRG RAM bank
- CPU $8000-$9FFF: 8 KB switchable PRG ROM/RAM bank
- CPU $A000-$BFFF: 8 KB switchable PRG ROM/RAM bank
- CPU $C000-$DFFF: 8 KB switchable PRG ROM/RAM bank
- CPU $E000-$FFFF: 8 KB switchable PRG ROM bank

CHR mode 0

- PPU $0000-$1FFF: 8 KB switchable CHR bank

CHR mode 1

- PPU $0000-$0FFF: 4 KB switchable CHR bank
- PPU $1000-$1FFF: 4 KB switchable CHR bank

CHR mode 2

- PPU $0000-$07FF: 2 KB switchable CHR bank
- PPU $0800-$0FFF: 2 KB switchable CHR bank
- PPU $1000-$17FF: 2 KB switchable CHR bank
- PPU $1800-$1FFF: 2 KB switchable CHR bank

CHR mode 3

- PPU $0000-$03FF: 1 KB switchable CHR bank
- PPU $0400-$07FF: 1 KB switchable CHR bank
- PPU $0800-$0BFF: 1 KB switchable CHR bank
- PPU $0C00-$0FFF: 1 KB switchable CHR bank
- PPU $1000-$13FF: 1 KB switchable CHR bank
- PPU $1400-$17FF: 1 KB switchable CHR bank
- PPU $1800-$1BFF: 1 KB switchable CHR bank
- PPU $1C00-$1FFF: 1 KB switchable CHR bank

Registers

Sound ($5000-$5015)

For details on sound operation, see MMC5 audio

NES internal state monitoring

All of these registers overlay various registers that are already used inside the NES, and are fully decoded. A game could write to a mirror of a PPU register to get the MMC5 out of sync, but it's not clear how that could be useful.

8x16 mode enable ($2000 = PPUCTRL)

- Written-by-read only
- Reset detection not used

```
7  bit  0
---- ----
xxZx xxxx
  |
  +------- Sprite size (0: 8x8 pixels; 1: 8x16 pixels)
```

Only when Z is set and at least one E bit is set does the MMC5 draw 8x16 sprites from eight independent banks.[2]

PPU Data Substitution Enable ($2001 = PPUMASK)

- Write only
- Reset detection not used

```
7  bit  0
---------
xxxE Exxx
   | |
   +-+---- 1,2,3: Substitutions enabled; 0: substitutions disabled
```

The MMC5 listens to writes to PPUMASK ($2001). When it sees that both E bits are cleared, it disables its ability to make substitutions on the PPU data bus. This includes disabling:

- Independent bank 8x16 sprite mode
- Extended attribute mode
- Vertical split mode

The MMC5 only listens to the fully decoded address $2001, so this can be tested by using the PPU's mirrors of this register. For example, writing to register $2009 will be heard by the PPU but not the MMC5. It is not clear if that could have a practical use beyond test purposes.

When the MMC5 sees $00 written to $2001, and then the PPU's rendering gets enabled via a mirror of $2001, the MMC5 still counts scanlines and can generate scanline interrupts even though it thinks $2001 is still disabled. The transition from disabled to enabled resets the scanline counter.

Driving pin 92 low similarly disables substitutions. Because no scanline interrupts occur with pin 92 held low, it seems to hold the scanline counter in reset.

Scanline Counter Reset ($2002 = PPUSTATUS, read only)

- Written-by-read only
- Reset detection not used

```
7  bit  0
---------
Exxx xxxx
|
+--------- 1: trigger scanline counter reset; 0: no effect
```

Unknown ($2005 = PPUSCROLL)

- Write only
- Reset detection is used

```
7  bit  0
---------
xxxx x000  Reset Value
---- ----
xxxx xYYY
      |||
      +++- CHR A2,A1,A0
```

The letterless MMC5 captures the low 3 bits of the second write to PPUSCROLL into a 3-bit counter. The value of the counter is output onto CHR A2,A1,A0 at certain times. It is not known how or when the counter does that. Tests using a mirror of PPUSCROLL on MMC5A were expected to cause rolled tiles, but that result was observed not to be true.

Second write tracking appears to be resynced each time the register $5130 is written.

Unknown ($2006 = PPUADDR, MMC5A only)

Power analysis has detected that the MMC5A monitors writes here, purpose unknown.

Scanline Counter Reset ($4014 = OAMDMA)

Writes to OAMDMA trigger a reset of the scanline counter. The value written is not captured by the MMC5.

Configuration

PRG mode ($5100)

- Write only
- Reset detection is used

```
7  bit  0
---- ----
xxxx xx11  Reset Value
---- ----
xxxx xxPP
       ||
       ++- Select PRG banking mode
```

- 0 - One 32KB bank
- 1 - Two 16KB banks
- 2 - One 16KB bank ($8000-$BFFF) and two 8KB banks ($C000-$DFFF and $E000-$FFFF)

- 3 - Four 8KB banks

Castlevania III uses mode 2, which is similar to VRC6 PRG banking. All other games use mode 3. The Koei games never write to this register, apparently relying on the MMC5 defaulting to mode 3 at power on.

CHR mode ($5101)

- Write only
- Reset detection not used

```
7  bit  0
---- ----
xxxx xxCC
       ||
       ++- Select CHR banking mode
```

- 0 - 8KB CHR pages
- 1 - 4KB CHR pages
- 2 - 2KB CHR pages
- 3 - 1KB CHR pages

Metal Slader Glory uses 4KB CHR pages. All other games use 1KB pages.

PRG RAM Protect 1 ($5102)

- Write only
- Reset detection is used

```
7  bit  0
---- ----
xxxx xx11  Reset Value
---- ----
xxxx xxWW
       ||
       ++- RAM protect 1
```

In order to enable writing to PRG RAM, this must be set to binary '10' (e.g. $02).

PRG RAM Protect 2 ($5103)

- Write only
- Reset detection is used

```
7  bit  0
---- ----
xxxx xx11  Reset Value
---- ----
xxxx xxWW
       ||
       ++- RAM protect 2
```

In order to enable writing to PRG RAM, this must be set to binary '01' (e.g. $01).

Internal extended RAM mode ($5104)

- Write only
- Reset detection is used

- MMC5 additionally keeps track whether this register has ever been written since reset, purpose unknown.

```
7  bit  0
---- ----
xxxx xx11  Reset Value
---- ----
xxxx xxXX
       ||
       ++- Specify extended RAM usage
```

$5104	CPU Access ($5C00-$5FFF) during blanking	CPU Access ($5C00-$5FFF) during rendering	PPUDATA Access ($2000-$2FFF)(1) during blanking	Available as Nametable	Enable Extended Attribute Mode
%00	No(3)	Write Only(4)	Read/Write	Yes	No
%01	No(3)	Write Only(4)	Read/Write	Yes[2]	Yes
%10	Read/Write	Read/Write	No	No	No
%11	Read Only	Read Only	No	No	No

(1)When configured as a nametable in register $5105, read and write access is possible through the PPU via registers $2006/$2007 when the PPU is not rendering. Based on register $5105, any nametables mapped to extended RAM will support the corresponding PPU address ranges for reads and writes. For example, if NT4 is mapped to extended RAM, reads/writes to PPU $2C00 will correspond with extended RAM $5C00.

(2)Though it is possible to still assign the extended RAM as a nametable in the mode %01, you are going to have the same data used twice, once as extended attribute data, and once as nametable data, this does not seem to be a useful combination. In the other modes, the nametable will read as if it contains all $00s.

(3)Counterintuitively, writes in these modes are only allowed when the PPU is rendering. If writes are attempted during V-blank, they may be ignored or cause a corruption at that memory address. In practice, temporarily switch to mode %10 if you wish to write during V-blank.

(4)Attempting to read extended RAM in those modes returns open bus.

Extended attributes

In mode %01, "Extended Attribute Mode", each byte of the MMC5's internal extended RAM is used to enhance the background tile at the corresponding nametable address. The extended attributes are 1-screen mirrored; in other words, they apply the same for all nametables.

Format of each extended attribute byte:

```
7  bit  0
---- ----
AACC CCCC
|||| ||||
||++-++++- Select 4 KB CHR bank to use with specified tile
++-------- Select palette to use with specified tile
```

In extended attribute mode, CHR banking behaves differently than normal when fetching background tiles from pattern tables:

- CHR mode (register $5101) is ignored. CHR banks are always 4KB in this mode.
- The values of the CHR banking registers $5120-$512B are also ignored.
- Bits 0-5 specified here are used for selecting a 4KB CHR bank on a per-tile basis.

- The two bits in $5130 are used globally as CHR bank bits 6 and 7.
- Driving pin 92 low disables extended attribute mode. Extended attribute mode is also automatically disabled based on PPUMASK monitoring. (See section on $2001 monitoring.) In these cases, the non-extended attribute table is used instead.

In other words, this works as if the nametable was extended from 8-bit to 14-bit tile offsets, with the ExRAM holding the upper 6-bits and the 2-bit palette value, while the nametable selected through $5105 contains the lower 8 bits.

Just Breed, *Yakuman Tengoku*, and the Koei games use extended attributes continuously.

Nametable mapping ($5105)

- Write only
- Reset detection not used

```
7  bit  0
---- ----
DDCC BBAA
|||| ||||
|||| ||++- Select nametable at PPU $2000-$23FF
|||| ++--- Select nametable at PPU $2400-$27FF
||++------ Select nametable at PPU $2800-$2BFF
++-------- Select nametable at PPU $2C00-$2FFF
```

Nametable values:

- 0 - CIRAM page 0
- 1 - CIRAM page 1
- 2 - Internal extended RAM
  - When $5104 is set to mode %10 or %11, the nametable will read as all zeros. This does not share functionality with fill mode.
- 3 - Fill-mode data
  - See registers $5106 and $5017

Examples:

Mode	Value	NTD	NTC	NTB	NTA
Vertical arrangement	$50	%01	%01	%00	%00
Horizontal arrangement	$44	%01	%00	%01	%00
Single-screen CIRAM 0	$00	%00	%00	%00	%00
Single-screen CIRAM 1	$55	%01	%01	%01	%01
Single-screen ExRAM	$AA	%10	%10	%10	%10
Single-Screen Fill-mode	$FF	%11	%11	%11	%11
Diagonal	$14	%00	%01	%01	%00

Fill-mode tile ($5106)

- Write only
- Reset detection not used

When a nametable is mapped to fill-mode in register $5105, all nametable fetches get replaced by the value of this register. Only the nametable is affected by fill mode. When the PPU later uses this information to fetch the corresponding tile from the pattern table, CHR banking is unaffected and continues to work normally. §

Fill-mode color ($5107)

- Write only
- Reset detection not used

```
7  bit  0
---- ----
xxxx xxAA
       ||
       ++- Specify background palette index to use for fill-mode nametable
```

When a nametable is mapped to fill-mode in register $5105, and $5104 is not in mode %01, all attribute table fetches get replaced by the value of this register. Each byte of the attribute table normally contains four 2-bit palette indexes. The two bits in this register are copied for all four indexes.

When $5104 is in mode %01, extended attribute mode does apply for fill mode, and this register is ignored. However, if pin 92 is driven low in this mode, extended attribute mode becomes disabled and this register comes back into effect.

PRG Bankswitching ($5113-$5117)

- Write only
- Reset detection is used only for $5117

In general, when the CPU accesses an address that corresponds to the range of a PRG bank designated by the present PRG mode, the bits of that PRG bank register are applied to the appropriate PRG address buses as follows:

```
7  bit  0
---- ----
1111 1111  Reset Value ($5117 only)
---- ----
RAAA AaAA
|||| ||||
|||| |||+- PRG ROM/RAM A13
|||| ||+-- PRG ROM/RAM A14
|||| |+--- PRG ROM/RAM A15, also selecting between PRG RAM /CE 0 and 1
|||| +---- PRG ROM/RAM A16
|||+------ PRG ROM A17
||+------- PRG ROM A18
|+-------- PRG ROM A19
+--------- RAM/ROM toggle (0: RAM; 1: ROM) (registers $5114-$5116 only)
```

Bank register effective areas versus PRG mode:

<table>
<tr><th></th><th colspan="4">CPU memory affected for each mode (see #PRG mode ($5100))</th></tr>
<tr><th>CPU Address</th><th>Mode 3 Registers</th><th>Mode 2 Registers</th><th>Mode 1 Registers</th><th>Mode 0 Registers</th></tr>
<tr><td>$6000-7FFF</td><td>$5113 (RAM only)</td><td>$5113 (RAM only)</td><td>$5113 (RAM only)</td><td>$5113 (RAM only)</td></tr>
<tr><td>$8000-9FFF</td><td>$5114</td><td rowspan="2">$5115</td><td rowspan="2">$5115</td><td rowspan="4">$5117 (ROM only)</td></tr>
<tr><td>$A000-BFFF</td><td>$5115</td></tr>
<tr><td>$C000-DFFF</td><td>$5116</td><td>$5116</td><td rowspan="2">$5117 (ROM only)</td></tr>
<tr><td>$E000-FFFF</td><td>$5117 (ROM only)</td><td>$5117 (ROM only)</td></tr>
</table>

Register $5113 always maps RAM and bits 7, 6, 5, and 4 are always ignored. Based on the letterless MMC5 die image, $5117.7 is fully implemented, but testing has confirmed that the bit is prevented from ever entering RAM mode for an unknown reason. Since $5117 appears to only map ROM, it is not possible to map the interrupt vectors to RAM in any mode. All known games have their reset vector in the last bank of PRG ROM, and the vector points to an address greater than or equal to $E000. This tells us that $5117 must have a reliable power-on value of $FF.

When a bankswitch register controls an 8kByte CPU address range, register bits 6..0 correspond to PRG A19..A13.

In PRG modes where a register controls a 16kByte CPU address range, register bits 6..1 correspond to PRG A19..A14. Register bit 0 is ignored and instead CPU A13 directly controls PRG A13. For example, comparing mode 3 to mode 2 for CPU address range $8000-BFFF. These are equivalent:

- In mode 3, write value $90 to $5114 and value $91 to $5115.
- In mode 2, write value $90 to $5115.

However, these are not equivalent:

- In mode 3, write value $91 to $5114 and value $92 to $5115.
- In mode 2, write value $91 to $5115.

The MMC5 ignores register bit 0 in the 16kByte bank.

Similarly, when register $5117 controls a 32kByte CPU address range in mode 0, register bits 6..2 correspond to PRG A19..A15, register bits 1..0 are ignored, and CPU A14..A13 directly control PRG A14..A13

Separate PRG-ROM and PRG-RAM Address Buses

The MMC5 has separate sets of address pins for PRG-ROM and PRG-RAM. This is a concept that has proven difficult to understand and explain. It all stems from the fact that CPU A15 is not directly routed to the cartridge; the signal /ROMSEL is supplied instead. Though it is possible to figure out CPU A15 from /ROMSEL, it takes extra time. This reduces the setup time where the address bus is valid before enabling the RAM and ROM /CEs. To avoid this, the MMC5 *ignores* /ROMSEL (and therefore CPU A15) when it comes to the PRG address, and /ROMSEL only figures into the logic of PRG ROM and RAM /CEs. This effectively creates a mirror on the PRG address buses for CPU address range $0000-7FFF and $8000-FFFF. Specifically, this makes range $6000-7FFF indistinguishable from $E000-FFFF. Because the MMC5 wanted to have separately controllable mapping for those ranges without reducing the setup time, its solution was to make two separate PRG address buses.

The MMC5's PRG-ROM address pins follow this logic. Though the PRG-ROM address bus is decoded at *all* CPU addresses, the gray areas always have PRG ROM /CE disabled:

CPU Address	Mode 3 PRG-ROM Address Source	Mode 2 PRG-ROM Address Source	Mode 1 PRG-ROM Address Source	Mode 0 PRG-ROM Address Source
$0000-1FFF	$5114	$5115	$5115	$5117
$2000-3FFF	$5115			
$4000-5FFF	$5116	$5116	$5117	
$6000-7FFF	$5117	$5117		
$8000-9FFF	$5114	$5115	$5115	$5117
$A000-BFFF	$5115			
$C000-DFFF	$5116	$5116	$5117	
$E000-FFFF	$5117	$5117		

The MMC5's PRG-RAM address pins follow this logic, again with PRG-RAM /CE always disabled in the gray areas:

CPU Address	Mode 3 PRG-RAM Address Source	Mode 2 PRG-RAM Address Source	Mode 1 PRG-RAM Address Source	Mode 0 PRG-RAM Address Source
$0000-1FFF	$5114	$5115	$5115	$5113*
$2000-3FFF	$5115			
$4000-5FFF	$5116	$5116	$5113*	
$6000-7FFF	$5113	$5113	$5113*	$5113*
$8000-9FFF	$5114	$5115	$5115	$5113*
$A000-BFFF	$5115			
$C000-DFFF	$5116	$5116	$5113*	
$E000-FFFF	$5113	$5113		

*In these larger areas of $5113, the selected 8kByte bank in $5113 is mirrored to fill the region.

If we overlap these two tables, the original table reemerges. Pink always has /CE enabled for RAM (using $5113), Blue always has /CE enabled for ROM (using $5117), and Purple chooses which /CE using the register bit 7. Gray always has RAM /CE and ROM /CE disabled at all times.

CPU Address	Mode 3 PRG Address Source	Mode 2 PRG Address Source	Mode 1 PRG Address Source	Mode 0 PRG Address Source
$0000-1FFF	$5114	$5115	$5115	$5113/$5117
$2000-3FFF	$5115			
$4000-5FFF	$5116	$5116	$5113/$5117	
$6000-7FFF	$5113/$5117	$5113/$5117	$5113/$5117	$5113/$5117
$8000-9FFF	$5114	$5115	$5115	$5113/$5117
$A000-BFFF	$5115			
$C000-DFFF	$5116	$5116	$5113/$5117	
$E000-FFFF	$5113/$5117	$5113/$5117		

PRG-RAM configurations

In commercial configurations, bits 0 and 1 select pages *within* an SRAM chip, and bit 2 selects between two separate SRAMs. 8K and 32K games have a single SRAM chip that will only be active when bit 2 is clear. 16K games instead have two chips, but only the first is battery backed.

Configuration	bank value & 7							
	0	1	2	3	4	5	6	7
ELROM 0K	open bus	open bus	open bus	open bus	open bus	open bus	open bus	open bus
EKROM 8K (1 x 8K)	0:$0000	0:$0000	0:$0000	0:$0000	open bus	open bus	open bus	open bus
ETROM 16K (2 x 8K)	**0**:$0000	**0**:$0000	**0**:$0000	**0**:$0000	**1**:$0000	**1**:$0000	**1**:$0000	**1**:$0000
EWROM 32K (1 x 32K)	0:$**0000**	0:$**2000**	0:$**4000**	0:$**6000**	open bus	open bus	open bus	open bus
Superset 64K (2 x 32K)	**0**:$**0000**	**0**:$**2000**	**0**:$**4000**	**0**:$**6000**	**1**:$**0000**	**1**:$**2000**	**1**:$**4000**	**1**:$**6000**

Since iNES headers were lacking reliable PRG-RAM size information before NES 2.0, some emulators may have selected these behaviours through ROM CRC checks.

Because no ExROM game is known to write PRG-RAM with one bank value and then attempt to read back the same data with a different bank value, emulating the PRG-RAM as 64K at all times can be used as a compatible superset for all games.

Investigation of the MMC5 pinout in 2018 revealed that bits 2 and 3 also control additional PRG-RAM address pins, which could theoretically have been used to select 16 banks of a single 128K SRAM, with bit 2 controlling PRG A15 directly rather than using the two chip select /CE outputs.

Other PRG-RAM notes

- *Bandit Kings of Ancient China* maps PRG-RAM to the CPU $8000+ area and expects to be able to write to it through there. Failure to emulate this causes corruption when the background is restored on the world map.
- *Uncharted Waters* requires emulating bankswitching of PRG-RAM: it writes to PRG-RAM at one CPU address and expects to be able to read the same data back via a different CPU address.
- Games with 16K PRG-RAM only battery-save the first 8K.
- List of MMC5 games which include a battery (http://bootgod.dyndns.org:7777/search.php?ines=5&battery=Yes&group=groupid).

CHR Bankswitching ($5120-$5130)

When using 8x8 sprites, only registers $5120-$5127 are used. Registers $5128-$512B are completely ignored.

When using 8x16 sprites, the PPU ignores the sprite pattern table address and can select tiles from the entire 8 KiB of pattern tables, which on other mappers overlaps with the background pattern table. The MMC5 keeps track of whether the PPU is fetching background tiles or sprite tiles, and has new registers to specify independent banks for the background tiles even if they appear to be the same address from the PPU. This effectively creates a CHR window of 12 KiB, with up to eight 1 KiB banks of sprites available simultaneously. Registers $5120-$5127 specify banks for sprites, registers $5128-$512B apply to background tiles, and the last set of registers written to (either $5120-$5127 or $5128-$512B) will be used for I/O via PPUDATA ($2007). [1] (https://forums.nesdev.org/viewtopic.php?p=193069#p193069) [2] (http://forums.nesdev.org/viewtopic.php?p=194120#p194120) The MMC5 knows that sprite data is being fetched by counting the number of fetches since the last detected scanline start, similar to how it detects the position for the vertical split.

Bandit Kings of Ancient China and *Uchuu Keibitai SDF* have non-pattern data stored in CHR ROM, read out via $2007.

The MMC5 is known to listen to the same address as the PPU to find out when to enable the 8x16 sprite mode; see above.

CHR select $5120-$512B

- Write only
- Reset detection not used

	PPU memory affected for each mode (see #CHR mode ($5101))			
Register	**1 KiB**	**2 KiB**	**4 KiB**	**8 KiB**
$5120	$0000-$03FF	none	none	none
$5121	$0400-$07FF	$0000-$07FF	none	none
$5122	$0800-$0BFF	none	none	none
$5123	$0C00-$0FFF	$0800-$0FFF	$0000-$0FFF	none
$5124	$1000-$13FF	none	none	none
$5125	$1400-$17FF	$1000-$17FF	none	none
$5126	$1800-$1BFF	none	none	none
$5127	$1C00-$1FFF	$1800-$1FFF	$1000-$1FFF	$0000-$1FFF
$5128	$0000-$03FF and $1000-$13FF	none	none	none
$5129	$0400-$07FF and $1400-$17FF	$0000-$07FF and $1000-$17FF	none	none
$512A	$0800-$0BFF and $1800-$1BFF	none	none	none
$512B	$0C00-$0FFF and $1C00-$1FFF	$0800-$0FFF and $1800-$1FFF	$0000-$0FFF and $1000-$1FFF	$0000-$1FFF

Caution: Unlike the MMC1 and unlike PRG banking on the MMC5, the banks are always indexed by the *currently selected size*. When using 2kb, 4kb or 8kb bank sizes, the registers hold bank index of that larger size, and lower bits are *not* ignored.

Upper CHR Bank bits ($5130)

- Write only
- Reset detection is used

```
7  bit  0
---- ----
xxxx xx00  Reset Value
---- ----
xxxx xxBB
       ||
       ++- Upper bits for subsequent CHR bank writes
```

When the MMC5 is using 2KB/1KB CHR banks, only 512KB/256KB of CHR ROM can be selected using the previous registers. To access all 1024KB in those modes, first write the upper bit(s) to register $5130 and then write the lower bits to $5120-$512B.

The only ExROM game with CHR ROM larger than 256KB is *Metal Slader Glory*, which uses 4KB CHR banks and does not use extended attributes. In other words, no official game relies on this register, and most don't even initialize it.

In extended attribute mode ($5104 = 1), this register likely acts as a global, instantaneous bank selection that gets appended as the most significant 2 bits of *all* of the tile-specific CHR banks, selecting which 256KB of CHR ROM is to be used for all background tiles on the screen. It is unlikely that the extended RAM stores all 10 bits per write, like registers $5120-$512B.

Other Registers

Vertical Split Mode ($5200)

- Write only
- Reset detection is used

```
7  bit  0
---- ----
00x0 0000  Reset Value
---- ----
ESxW WWWW
|| | ||||
|| +-++++- Specify vertical split threshold tile count
|+-------- Specify vertical split region screen side (0:left; 1:right)
+--------- Enable vertical split mode (0:disabled; 1:enabled)
```

Uchuu Keibitai SDF uses split screen mode during the intro, where it shows ship stats. *Bandit Kings of Ancient China* uses split screen mode during the ending sequence[3].

MMC5 internal extended RAM is always used as the nametable in split screen mode. Extended RAM mode ($5104) should be set to %00 or %01. RAM modes %10 or %11 disable split mode. Driving pin 92 low also disables split mode. Split mode is automatically disabled based on PPUMASK monitoring. (See section on $2001 monitoring)

When using mode %01 with vertical split mode, the non-split region gains extended attributes, and the split region does not use extended attributes. As long as the non-split region remains horizontally scrolled to align with the left edge of a nametable, it is possible to share the extended RAM for both purposes. For example, a split region on the right side of the screen will read its nametable data and non-extended attribute data from the “right side” of the extended RAM, and the left side of the screen will read its nametable data from wherever it is assigned in $5105, and get its extended attributes from the “left side” of the extended RAM. The two separate functions of extended RAM data would not overlap this way.

Method of Operation

Based on nametable fetch count (horizontally) and scanline count (vertically), the MMC5 makes substitutions to the nametable data, attribute table data, and the lowest 3 address bits of the pattern data.

Horizontal Behavior

For each visible scanline, the PPU fetches nametable data for 34 background tiles. The MMC5 locates the split threshold by counting these fetches and comparing the count to the count value stored in register $5200. When the PPU is rendering in the split region, the MMC5 replaces the normal nametable data with the split region nametable data. Because the split threshold is located based on nametable fetch count and not based on the actual nametable address requested by the PPU, the location of the split is more of an absolute position on the screen, always positioned the same number of tiles from the left side of the screen. The PPU’s fine horizontal

scrolling can cause the first tile rendered to be of fractional width, however the vertical split threshold will always occur at a full tile boundary, so the exact location of the split threshold is affected by fine horizontal scrolling, potentially deviating by up to 7 pixels this way.

Left Split:

- Tiles 0 to T-1 are the split region.
- Tiles T and on are rendered normally.

Right Split:

- Tiles 0 to T-1 are rendered normally.
- Tiles T and on are the split region.

There is no intended horizontal scrolling of any kind for the split region. Right-side split regions will always remain right-aligned with the right-hand side of the nametable, and left-side split regions will always remain left-aligned with the left-hand side of the nametable. Coarse horizontal scrolling can still be used for the non-split region.

Vertical Behavior

The MMC5 is able to have its own vertical origin in the split region because the nametable data that it substitutes is based on scanline count, which is always aligned to the top of the screen.

The MMC5 has a separate v-split scanline counter that resets to the vertical scrolling value in $5201 each v-blank. It uses this to know what nametable data to substitute in the split region on each scanline. Though the PPU continues requesting nametable data corresponding to its normal vertical scrolling in the split region, the MMC5 is ignoring the nametable address requested in this case and directly substituting its own nametable data.

One additional caveat is fine vertical scrolling of the split region. For each tile on each scanline, the PPU reads from a nametable, and uses that information to fetch the pattern data for 1 row of pixels for that tile. The PPU normally keeps track of which row to fetch using PPU A0..3. However, if the MMC5's fine vertical scrolling doesn't match the PPU's fine vertical scrolling, it won't be fetching the correct row of pixels from the tile. This is why the MMC5 provides CHR A0..3 from the lowest 3 bits of the v-split scanline counter when rendering the split region. By default, Nintendo PCBs are wired in "CL Mode", which prevents the correct fine vertical scrolling of the split region. (See MMC5 pinout).

Vertical scrolling for the split region operates like normal vertical scrolling. 0-239 are valid scroll values, whereas 240-255 will display attribute table data as nametable data for the first few scanlines. The MMC5 does proper vertical mirroring of the split region nametable so that scrolling down properly rolls over to the top of the nametable, skipping the attribute table data that would naturally be located there.

Vertical Split Scroll ($5201)

- Write only
- Reset detection is used, value $00

```
All eight bits specify the vertical scroll value to use in split region
```

MMC5 boards wired in "CL" mode should only use vertical scroll values whose bottom 3 bits match the PPU's fine vertical scroll value. Using a mismatched value will cause tiles to seem to "roll" within themselves. In "SL" mode, any values can be used. (No existing games used the SL board configuration.)

Horizontal scrolling is not allowed within the split region.

Vertical Split Bank ($5202)

- Write only
- Reset detection not used

```
All eight bits select a 4 KB CHR bank at $0000-$0FFF and $1000-$1FFF while rendering the split region.
```

IRQ Scanline Compare Value ($5203)

- Write only
- Reset detection not used

All eight bits specify the target scanline number at which to generate a scanline IRQ. Value $00 is a special case that will not produce IRQ pending conditions, though it is possible to get an IRQ while this is set to $00 (due to the pending flag being set already.) You will need to take additional measures to fully suppress the IRQ. See the detailed discussion.

Scanline IRQ Status ($5204, read/write)

- Read/Write
- Reset detection is used

Write

```
7  bit  0
---- ----
0xxx xxxx  Reset Value
---- ----
Exxx xxxx
|
+--------- Scanline IRQ Enable flag (1=enabled)
```

Read

```
7  bit  0
---- ----
SVxx xxxJ  MMC5A default power-on value = $00
||      |
||      +- Abnormal M2 vs CPU A0 sequence detection flag
|+-------- "In Frame" flag
+--------- Scanline IRQ Pending flag
```

The Scanline IRQ Pending flag becomes set at any time that the internal scanline counter matches the value written to register $5203. If the scanline IRQ is enabled, it will also generate /IRQ to the system.

The "In Frame" flag is set when the PPU is actively rendering visible scanlines and cleared when not rendering; for example, vertical blank scanlines. This flag does not clear for the short H-Blank period between visible scanlines. When pin 92 is driven low, this flag remains low at all times. Additionally, scanline IRQs no longer occur when pin 92 is driven low.

Bit 0 exists in both versions of MMC5. If an abnormal sequence of M2 vs CPU A0 occurs, there is a latch that stores this permanently until system reset detection. The letterless MMC5 additionally shows this latch on the CL3 pin. When no error has been detected, the bit is 0 and CL3 is high. Though the MMC5A has the same

detection and status bit, no way to enable the function on the CL3 pin has been found. The function is not triggered by a CPU crash when triggered via JAM opcode. The intention of the latch is not known, but it may be involved in detecting when the player turns off the power without holding reset.

Any time this register is read, the Scanline IRQ Pending flag is automatically cleared (acknowledging the IRQ). Acknowledging the IRQ does not clear the scanline counter, and you may write a larger value to $5203 within a scanline ISR in order to generate another IRQ on an additional scanline within the same frame.

For details, see § Scanline Detection and Scanline IRQ.

Unsigned 8x8 to 16 Multiplier ($5205, $5206 read/write)

- Write only
- Reset detection not used

The unsigned 16-bit product is available to be read from these registers immediately after writing. All 65536 combinations of multiplicand and multiplier were tested and verified correct on MMC5A here[3] (https://forums.nesdev.org/viewtopic.php?p=226537#p226537).

Write

- $5205 8-bit Unsigned Multiplicand
- $5206 8-bit Unsigned Multiplier

Read

- $5205 Unsigned 16-bit Product (low byte)
- $5206 Unsigned 16-bit Product (high byte)

Internal extended RAM ($5C00-$5FFF, read/write)

Refer to register $5104 for special behaviors of the MMC5's 1KB internal extended RAM.

MMC5A

The MMC5A was a later revision of MMC5 that included some extra features. Other than Just Breed mysteriously writing to $5800, no game is known that uses these features. Therefore, these will probably tend to have poor support from emulators.

MMC5A Registers

Registers $5207, $5208, $5209, $520A, and range $5800-$5BFF are present only in MMC5A.

CL3 / SL3 Data Direction and Output Data Source (MMC5A: $5207 write only)

```
7  bit  0
---- ----
ABxx xxCD  MMC5A default power-on write value = 11xx xx00
||     ||
||     |+- MMC5.97 (CL3) Function (0 = I/O Control, 1 = $5800 read control
||     +-- MMC5.98 (SL3) Function (0 = I/O Control, 1 = $5800 write control
|+-------- MMC5.97 (CL3) I/O Data Direction (0 = output, 1 = input)
+--------- MMC5.98 (SL3) I/O Data Direction (0 = output, 1 = input)
```

When a function bit is set to 1, the pin becomes controlled by the applicable CPU writes or reads in the range $5800-$5BFF. The pin is normally driven high, and when the read or write occurs, it has a low pulse corresponding to !(M2). The intended purpose is unknown. It is theorized that it may attach a peripheral or additional RAM. When in this mode, none of the I/O configuration bits have any effect.

CL3 / SL3 Status (MMC5A: $5208 read/write)

Write

```
7  bit  0
---- ----
ABxx xxxx  MMC5A default power-on write value = 00xx xxxx
||
|+-------- Value to be output on MMC5.97 pin (CL3) if it is set as an I/O output in $5207.
+--------- Value to be output on MMC5.98 pin (SL3) if it is set as an I/O output in $5207.
```

Warning: The PCB may connect pins 97 and 98 directly to GND. Though not totally confirmed, setting them as output high while connected like this has appeared to break the output drivers of these pins. Also note that enabling the $5800 function will have these pins driving high most of the time as well.

Read

```
7  bit  0
---- ----
ABxx xxxx
||
|+-------- Input value of MMC5.97 pin (CL3)
+--------- Input value of MMC5.98 pin (SL3)
```

16-bit Hardware Timer with IRQ (MMC5A: $5209 read/write, $520A write)

Read

- $5209

```
7  bit  0
---- ----
Vxxx xxxx  MMC5A default power-on read value = $00
|
+--------- Hardware Timer IRQ Flag
```

Write

- $5209

```
7  bit  0
---- ----
TTTT TTTT  MMC5A default power-on write value = $00
|||| ||||
++++-++++- Timer count LSB
```

- $520A

```
7  bit  0
---- ----
TTTT TTTT  MMC5A default power-on write value = $00
```

```
|||| ||||
++++-++++- Timer count MSB
```

Based on findings from krzysiobal: The timer automatically starts when writing any value to register $5209, if the 16-bit timer value does not equal $0000. For example, to write value $0100, you would first write $01 (MSB) to register $520A, which does not start the timer. Then write $00 (LSB) to register $5209, which at that point will start the timer from value $0100.

Each 8-bit value is written directly to an internal 16-bit counter that decrements each CPU cycle, specifically on the rising edge of M2. Additional writes while the timer is running will directly overwrite that portion of the counter. Reading register $5209 while the timer is running reports $00. The transition from counter value $0001 to $0000 generates an IRQ and sets the hardware timer IRQ flag. The timer stops at this point. Reading this register reports the IRQ flag, then automatically clears the IRQ and IRQ flag.

If the MMC5 detects a reset, it clears the timer if active, and it clears the IRQ and IRQ flag if set. Reset detection works by looking for a gap larger than about 11 usec on M2.

This register's IRQ operation is completely independent from register $5204. Disabling interrupts through $5204 has no effect on, and reading $5204 does not report on, IRQs generated these registers.

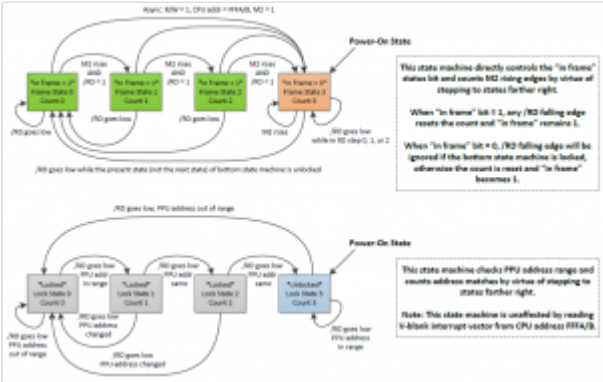
Unknown Address Range (MMC5A: $5800-$5BFF, write only)

Reads and writes in this address range are reflected on the CL3 and SL3 pins when register $5207 = $03. The purpose of this function is unknown. Minute VCC current spikes shortly after rising edge of M2 during writes in this range exhibit the same characteristics as writes in internal extended RAM range $5C00-$5FFF, suggesting possible existence of RAM in this range, though experimentally reading from this range is always met with open CPU bus.

Address $5800 is written to by Just Breed. During each V-Blank, it writes value $03, then $01 to this address, reads and writes to PPU registers, then writes value $00 to this address once complete. It is theorized that this was used as a debug signal to measure CPU usage / idle time during development.

Scanline Detection and Scanline IRQ

The MMC5 detects scanlines by first looking for three consecutive PPU reads from the same nametable address in the range $2xxx. Once this has been seen, the next PPU read, regardless of address, is the point at which the scanline is detected. This works because the PPU does two matching dummy nametable byte reads at the end of each scanline, followed by a third matching nametable byte read at the beginning of the next scanline, followed by an attribute table byte read. So, the scanline gets detected when the PPU does the attribute table byte read, which is at PPU cycle 4.



MMC5 'in frame' status bit state diagram

Once that occurs, if the "in-frame" flag (register $5204) was clear, it becomes set, and the internal 8-bit scanline counter is reset to zero; but if it was already set, the scanline counter is incremented, then compared against the value written to $5203. If they match, the "irq pending" flag is set.

The IRQ pending flag is raised when the desired scanline is reached *regardless* of whether or not the scanline IRQ is enabled, i.e. even after a 0 was written to the scanline IRQ enable flag. However, an actual IRQ is only sent to the CPU if both the scanline IRQ enable flag and IRQ pending flag are set. A $5203 value of $00 is a

special case where the comparison is never true. The MMC5's scanline IRQ occurs at PPU cycle 4, unlike the simpler scanline counter of the MMC3, which usually generates an IRQ around PPU cycle 260. See also [4] (http://forums.nesdev.org/viewtopic.php?t=7653).

The "in-frame" flag is cleared when the PPU is no longer rendering. This is detected when 3 CPU cycles pass without a PPU read having occurred (PPU /RD has not been low during the last 3 M2 rises). The PPU does this in these conditions:

- the PPU begins the post-render scanline 240
- the PPU stops rendering because the user program wrote to CPU address $2001 with bits 3 and 4 clear.
- note: the MMC5 may not listen directly to writes to $2001 for this behavior, or it may be monitoring for particular transitions of $2001 that have not yet been fully tested.

The "in frame" flag is cleared, scanline IRQ is automatically acknowledged, and the internal scanline counter is reset in any of these conditions:

- the V-blank NMI occurs, i.e. CPU reads the interrupt vector from addresses $FFFA and $FFFB
- the user program intentionally reads from CPU addresses $FFFA or $FFFB
- believed but not tested: the user program writes to $4014 OAMDMA
- the 241st scanline is detected.
- when the MMC5 believes that $2001 bits 3 and 4 are both clear and then either one of them gets set.

The scanline IRQ is acknowledged, but the "in frame" flag is not cleared and the scanline counter is not reset in any of these conditions:

- the user program reads register $5204
- scanline 0 is detected

When system reset detection occurs, the only thing that happens is scanline IRQ becomes disabled. All other operation continues unaffected. These things happen any time that scanline IRQ becomes disabled:

- if /IRQ was low due to scanline IRQ pending flag, /IRQ returns high
- the scanline IRQ pending flag remains unaffected
- register $5203 value remains unaffected
- scanline counter remains unaffected
- enabling the scanline IRQ will cause immediate /IRQ low if IRQ pending flag is set, but will not affect anything else

This means in pseudo-code:

```
(On every PPU read -- PPU /RD falling edge)
if address >= $2000 and address <= $2FFF and address == lastAddress
   matchCount := matchCount +1
   if matchCount == 2
     if inFrame == false
        inFrame := true
        scanline := 0
     else
        scanline := scanline +1
        if scanline == [$5203]
           irqPending := true
else
   matchCount := 0
lastAddress := address
ppuIsReading := true

(On every CPU cycle -- M2 rising edge)
if ppuIsReading
   idleCount := 0
else
   idleCount := idleCount +1
   if idleCount == 3
      inFrame := false
```

```
      lastAddress := undefined
ppuIsReading := false

(On every CPU write)
if address == $2001 and (value & $18) == 0
   inFrame := false
   lastAddress := undefined

(On every CPU read)
if address == $FFFA or address == $FFFB
   inFrame := false
   lastAddress := undefined
```

Please refer to the state diagrams on the right for a more formal description of the scanline and in-frame detection counters.

Hardware

The MMC5 exists in a 100-pin rectangular QFP package, see MMC5 pinout for details.

MMC5 cartridge PCBs can be configured to different modes, see ExROM for details.

At least two different versions of the MMC5 are known to exist: MMC5, and MMC5A. MMC5A has the addition of registers $5207, $5208, $5209, and $520A: SL3/CL3 control and hardware timer.

References

1. "Why Your Game Paks Never Forget". *Nintendo Power* #20 (March 1991), pp. 28-31.
2. krzysiobal's RE notes (https://forums.nesdev.org/viewtopic.php?p=229375#p229375)
3. https://forums.nesdev.org/viewtopic.php?f=2&t=12764

External links

- NES Mapper list by Disch [5] (http://www.romhacking.net/documents/362/)
- Nintendo MMC5 by goroh, translated by Sgt. Bowhack [6] (https://nesdev.org/mmc5-e.txt)
- Nintendo MMC5 Bankswitching by Kevin Horton [7] (https://nesdev.org/mmc5_bank_switch.txt)

Retrieved from "https://www.nesdev.org/w/index.php?title=MMC5&oldid=23553"

This page was last edited on 27 February 2026, at 23:00.